DSA Experiment 3

Title: Perform Bubble Sort, Selection Sort, Insertion Sort, Quick Sort, and Merge Sort using C

Name-Mokshada Hejib **PRN**-B25ET1013 **Div**-SY1 A

```
#include <stdio.h>
void bubbleSort(int arr[], int n)
{
    int i, j, temp;
    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
void selectionSort(int arr[], int n)
{
    int i, j, minindex, temp;
    for (i = 0; i < n - 1; i++)
    {
        minindex = i;
        for (j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[minindex])
                minindex = j;
        }
        temp = arr[i];
        arr[i] = arr[minindex];
        arr[minindex] = temp;
    }
}
void insertionSort(int arr[], int n)
{
    int i, j, key;
    for (i = 1; i < n; i++)
```

```
    {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
void swap(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}
int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;
    int j;
    for (j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}
void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int p = partition(arr, low, high);
        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
}
void merge(int arr[], int l, int mid, int h)
{
```

```
    int i = l;
    int j = mid + 1;
    int k = l;
    int temp[100];
    while (i <= mid && j <= h)
    {
        if (arr[i] < arr[j])
        {
            temp[k] = arr[i];
            i++;
        }
        else
        {
            temp[k] = arr[j];
            j++;
        }
        k++;
    }
    while (i <= mid)
    {
        temp[k] = arr[i];
        i++;
        k++;
    }
    while (j <= h)
    {
        temp[k] = arr[j];
        j++;
        k++;
    }
    for (i = l; i <= h; i++)
        arr[i] = temp[i];
}
void mergeSort(int arr[], int l, int h)
{
    if (l < h)
    {
        int mid = (l + h) / 2;

        mergeSort(arr, l, mid);
        mergeSort(arr, mid + 1, h);
        merge(arr, l, mid, h);
    }
}
```

```
void display(int arr[], int n)
{
    int i;
    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
int main()
{
    int arr[100], n, i, choice;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);

    for (i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    printf("\nOriginal array: ");
    display(arr, n);
    printf("\nChoose Sorting Method:\n");
    printf("1. Bubble Sort\n");
    printf("2. Selection Sort\n");
    printf("3. Insertion Sort\n");
    printf("4. Quick Sort\n");
    printf("5. Merge Sort\n");
    printf("\nEnter your choice: ");
    scanf("%d", &choice);
    switch (choice)
    {
        case 1:
            bubbleSort(arr, n);
            printf("\nBubble Sort selected.");
            break;

        case 2:
            selectionSort(arr, n);
            printf("\nSelection Sort selected.");
            break;
        case 3:
            insertionSort(arr, n);
            printf("\nInsertion Sort selected.");
            break;
        case 4:
```

```
            quickSort(arr, 0, n - 1);
            printf("\nQuick Sort selected.");
            break;
        case 5:
            mergeSort(arr, 0, n - 1);
            printf("\nMerge Sort selected.");
            break;
        default:
            printf("\nInvalid choice!");
            return 0;
    }
    printf("\nSorted array: ");
    display(arr, n);
    return 0;
}
```

OUTPUT:

Output

```
Enter number of elements: 3
Enter 3 elements:
4
56
54

Original array: 4 56 54

Choose Sorting Method:
1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort

Enter your choice: 1

Bubble Sort selected.
Sorted array: 4 54 56
```

Output

```
Enter number of elements: 3
Enter 3 elements:
67
55
42

Original array: 67 55 42

Choose Sorting Method:
1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort

Enter your choice: 2

Selection Sort selected.
Sorted array: 42 55 67
```

Output

```
Enter number of elements: 3
Enter 3 elements:
23
2
44

Original array: 23 2 44

Choose Sorting Method:
1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort

Enter your choice: 3

Insertion Sort selected.
Sorted array: 2 23 44
```

Output

```
Enter number of elements: 4
Enter 4 elements:
34
2
6
43

Original array: 34 2 6 43

Choose Sorting Method:
1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort

Enter your choice: 4

Quick Sort selected.
Sorted array: 2 6 34 43
```

Output

```
Enter number of elements: 3
Enter 3 elements:
32
5
34

Original array: 32 5 34

Choose Sorting Method:
1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort

Enter your choice: 5

Merge Sort selected.
Sorted array: 5 32 34
```